

Refatoração do site Tô de Voluntário

Relatório técnico — Bootstrap, acessibilidade, arquitetura e preparação do banco de dados (Supabase)

1. Objetivo

Reorganizar a arquitetura de pastas do projeto, adotar Bootstrap 5 + Bootstrap Icons, implementar acessibilidade real (leitor de tela, Libras, alto contraste, daltonismo), substituir emojis por ícones, corrigir bugs encontrados durante o processo e deixar o ambiente pronto para integrar o Supabase — sem alterar layout, textos ou informações já existentes no site.

2. Arquitetura de pastas

Antes, `style.css` e `script.js` ficavam soltos na raiz. Agora:

```
assets/  
  css/style.css  
  js/script.js  
  js/acessibilidade.js  
  js/supabase-client.js  
paginas/*.html  
supabase/migrations/*.sql  
documentacao/*.html  
.env / .env.example / .gitignore
```

Os arquivos foram movidos com `git mv` para preservar o histórico do Git.

3. Bootstrap e ícones

Bootstrap 5.3.3 e Bootstrap Icons 1.11.3 foram incluídos via CDN em todas as 9 páginas, carregados **antes** do `style.css` próprio do site — assim, qualquer regra existente continua tendo prioridade na cascata CSS e o layout visual não muda. Bootstrap foi usado para o painel de acessibilidade (offcanvas, switches, selects) e para reforçar semântica/estados de foco, não para reescrever o grid ou os componentes já existentes.

Todos os emojis usados como ícone (👤 ⚙️ 🗣️ 🔔 🔒 ❓ 🏠 🔑 ❤️ etc.) foram trocados por `<i class="bi bi-...">`, com `aria-hidden="true"` quando o ícone é apenas decorativo (o texto ao lado continua sendo a informação real) e `aria-label` nos poucos casos em que o ícone é o único conteúdo do botão (fechar sidebar, abrir menu).

4. Módulo de acessibilidade (`assets/js/acessibilidade.js`)

Único ponto que lê/grava as preferências no `localStorage` e aplica as classes correspondentes. Qualquer controle da página (painel flutuante ou aba "Acessibilidade" das Configurações) usa o atributo `data-acess="chave"` e é sincronizado automaticamente — sem duplicar lógica em cada página.

- **Leitor de tela / navegação por teclado:** skip-link ("Pular para o conteúdo principal") em todas as páginas, `aria-label` em landmarks, foco visível reforçado (`:focus-visible`), e associação correta de `<label for>` com `id` em todos os campos de formulário do site.
- **Libras:** integração do widget oficial do governo federal (VLibras, `vlibras.gov.br`), presente em todas as páginas, com um atalho no painel de acessibilidade que abre o intérprete.
- **Alto contraste, modo escuro, tamanho de fonte e espaçamento entre linhas:** já existiam no site, mas estavam duplicados e parcialmente quebrados (ver seção de bugs); foram centralizados na aba "Acessibilidade" das Configurações.
- **Daltonismo:** filtros SVG (`feColorMatrix` , matrizes de Machado/Oliveira/Fialho, 2009) para protanopia, deuteranopia, tritanopia e acromatopsia, selecionáveis no painel.

5. Bugs encontrados e corrigidos durante os testes reais no navegador

O site foi aberto de fato num navegador (servidor HTTP local) e testado como um usuário cego (só teclado + árvore de acessibilidade), usuário de alto contraste, daltônico e usuário de Libras. Isso revelou problemas que só aparecem em uso real:

Problema	Causa	Correção
Erro de JavaScript em todas as páginas (Identifier 'supabase' already declared)	O SDK do Supabase (CDN) já usa o nome global <code>supabase</code> ; o <code>supabase-client.js</code> também declarava uma variável com esse nome.	Variável renomeada para <code>supabaseClient</code> .
Switches (modo escuro, alto contraste etc.) do painel flutuante ficavam invisíveis/fora da tela	Uma regra própria (<code>padding: 10px 0</code>) zerava o <code>padding-left</code> que o Bootstrap usa para posicionar o "botão" do switch.	Regra ajustada para não mexer no eixo horizontal (<code>padding-top</code> / <code>padding-bottom</code> apenas).
Alto contraste não cobria cards e containers internos das páginas	A regra original só mudava <code>body</code> , links e campos de formulário.	Adicionada uma regra abrangente (<code>body.alto-contraste *</code>) que força fundo preto/texto branco em qualquer elemento, garantindo cobertura total.
Checkbox "Alto contraste" da aba Acessibilidade não fazia nada	Existia no HTML mas nunca foi ligado a nenhum código JavaScript (era um controle "morto").	Ligado ao mecanismo <code>data-acess</code> compartilhado.
Página de Configurações tinha duas implementações da troca de abas rodando ao mesmo tempo (uma no <code>script.js</code> , outra em um <code><script></code> inline)	Duplicação de código acumulada ao longo do desenvolvimento.	Removida a versão inline; sobrou uma única fonte de verdade.

6. Limpeza e organização do código

- Removidos 4 blocos de `<script>` inline que reimplementavam (de forma divergente) o mesmo comportamento do menu lateral já coberto por `assets/js/script.js`.
- Lógicas únicas de cada página (ex.: modal de termos em `cadastro-voluntario.html`) foram preservadas.
- Comentários genéricos/sem função foram removidos; os comentários de seção que ajudam a navegar o arquivo (`<!-- HEADER -->`, `<!-- RODAPÉ -->`) foram mantidos.

7. Dados dinâmicos (API do IBGE)

O filtro de estado/cidade em `oportunidades.html` usava uma lista fixa de 5 estados. Foi trocado por chamadas à API pública do IBGE (`servicodados.ibge.gov.br`), carregando os 27 estados e as cidades reais de cada um, sem custo e sem chave de API.

8. Segurança e preparação do Supabase

- Projeto Supabase configurado (`xeokfq1khrynxyzldwvup`). A chave **anon/publishable** foi colocada em `assets/js/supabase-client.js` — ela é feita para ficar no navegador, e a segurança real vem do Row Level Security (RLS) do banco.
- As chaves **secret** e **service_role** (acesso total, ignoram RLS) foram isoladas em `.env`, que está no `.gitignore` e nunca deve ir para o front-end nem para o repositório Git. Criado também um `.env.example` sem valores reais, para documentar quais variáveis existem.

Atenção: essas chaves foram compartilhadas em texto simples durante a conversa. Elas já foram tratadas com o devido cuidado nos arquivos do projeto, mas, como boa prática, vale considerar rotacioná-las no painel do Supabase caso o histórico desta conversa possa ser acessado por outras pessoas.

9. Banco de dados (`supabase/migrations/20260812000000_criar_estrutura_inicial.sql`)

Schema relacional completo, com nomes de tabelas e campos em português, refletindo os formulários reais do site:

- **perfis** (estende `auth.users` do Supabase Auth), **voluntarios**, **organizacoes** (ONGs) — dados sensíveis (ex.: pessoa com deficiência) isolados em `voluntarios`, fora do perfil básico.
- **causas** e **habilidades** — tabelas de apoio (em vez de texto livre) para os filtros de oportunidades.
- **oportunidades**, **oportunidade_habilidades** (relação N:N) e **candidaturas** (inscrição de um voluntário numa vaga).
- **doacoes**, **mensagens_contato** e **agendamentos** — cobrindo os formulários de Doações e Contato.

Todas as tabelas têm Row Level Security habilitada, com políticas específicas (ex.: um voluntário só vê suas próprias candidaturas; oportunidades encerradas não aparecem para o público; formulários de contato/agendamento aceitam envio de visitantes não autenticados, mas só a organização consegue ler as respostas).

10. Testes realizados

Site servido localmente (HTTP) e testado num navegador real, cobrindo:

- Navegação 100% por teclado (Tab, skip-link, Enter/Espaço em controles customizados);
- Árvore de acessibilidade (equivalente ao que um leitor de tela anuncia) em todas as páginas com formulário — nenhum campo órfão encontrado após as correções;
- Alto contraste, modo escuro, tamanho de fonte e espaçamento, ligados e desligados visualmente;
- Os 4 filtros de daltonismo, aplicados e revertidos;
- Abertura do intérprete de Libras (VLibras) a partir do painel;
- Carregamento real dos 27 estados via API do IBGE;
- Modal de termos de uso em `cadastro-voluntario.html`.

11. Limitações conhecidas e próximos passos

- O site continua sendo multi-página (não uma SPA com roteamento client-side). Dado o volume de formulários e páginas já prontos, essa foi a opção mais segura para não arriscar quebrar algo — se um SPA "de verdade" for necessário, é uma etapa separada e maior.
- O Supabase está **preparado**, mas os formulários (cadastro, login, oportunidades, contato) ainda não estão conectados ao banco — hoje eles não enviam dados a lugar nenhum.
- Os testes de acessibilidade foram feitos com a árvore de acessibilidade do Chrome, não com um leitor de tela real (NVDA/JAWS/VoiceOver); o comportamento costuma ser equivalente, mas vale uma validação manual antes de publicar.
- Os filtros de daltonismo são um recurso visual de apoio (mesmo padrão usado por diversos plugins de acessibilidade), não uma correção clínica.

Próximos passos sugeridos: conectar os formulários às tabelas do Supabase (auth + CRUD), validar com um leitor de tela real, popular `oportunidades / organizacoes` com dados reais, e decidir se compensa migrar para uma SPA.